

LAPORAN TUGAS PRAKTIKUM JOBSHEET 14

TREE

Nama : Jonathan Abdiel Haryono
Absen : 14
Kelas : TI 1G

TUGAS PRAKTIKUM

1. Buat method di dalam class **BinaryTree00** yang akan menambahkan node dengan cara rekursif (**addRekursif()**).
2. Buat method di dalam class **BinaryTree00** untuk menampilkan data mahasiswa dengan IPK paling kecil dan IPK yang paling besar (**cariMinIPK()** dan **cariMaxIPK()**) yang ada di dalam binary search tree.
3. Buat method dalam class **BinaryTree00** untuk menampilkan data mahasiswa dengan IPK di atas suatu batas tertentu, misal di atas 3.50 (**tampilMahasiswaIPKdiAtas(double ipkBatas)**) yang ada di dalam binary search tree.
4. Modifikasi class **BinaryTreeArray00** di atas, dan tambahkan :
 - method **add(Mahasiswa data)** untuk memasukkan data ke dalam binary tree
 - method **traversePreOrder()**

JAWABAN

1. **addRekursif(Node14 current, Mahasiswa14 mahasiswa)**

```
void addRekursif(Node14 current, Mahasiswa14 mahasiswa) { // menambahkan node dengan rekursif

    if (isEmpty()) {
        root = new Node14(mahasiswa);
        return;
    }

    if (mahasiswa.ipk < current.mahasiswa.ipk) { // kalau ipk yg mau diinput lebih kecil dari ipk saat ini
        if (current.left == null) {
            current.left = new Node14(mahasiswa);
        }
        else {
            addRekursif(current.left, mahasiswa);
        }
    }
    else if (mahasiswa.ipk > current.mahasiswa.ipk) {
        if (current.right == null) {
            current.right = new Node14(mahasiswa);
        }
        else {
            addRekursif(current.right, mahasiswa);
        }
    }
}
```

```

    }
  }
}

```

2. cariMinIPK()

```

Node14 cariMinIPK() {
    if (isEmpty()) {
        System.out.println("Tree masih kosong.");
        return null;
    }

    Node14 current = root;

    while (current.left != null) {
        current = current.left;
    }

    return current;
}

```

cariMaxIPK()

```

Node14 cariMaxIPK() {
    if (isEmpty()) {
        System.out.println("Tree masih kosong.");
        return null;
    }

    Node14 current = root;

    while (current.right != null) {
        current = current.right;
    }

    return current;
}

```

3. tampilMahasiswaIPKdiAtas(double ipkBatas, Node14 current)

```

void tampilMahasiswaIPKdiAtas(double ipkBatas, Node14 current) {

    if (isEmpty()) {
        System.out.println("Tree masih kosong.");
        return;
    }

    if (current != null) {

        tampilMahasiswaIPKdiAtas(ipkBatas, current.left);

        if (current.mahasiswa.ipk > ipkBatas) {

```

```

        current.mahasiswa.tampilInformasi();
    }

    tampilMahasiswaIPKdiAtas(ipkBatas, current.right);
}
}

```

4. **add(Mahasiswa14 data)**

```

void add(Mahasiswa14 data) {

    int index = 0;

    while (index < dataMahasiswa.length) {
        if (dataMahasiswa[index] == null) {
            dataMahasiswa[index] = data;

            if (index > idxLast) {
                idxLast = index;
            }

            return;
        }

        if (data.ipk < dataMahasiswa[index].ipk) {
            index = 2 * index + 1;
        }
        else {
            index = 2 * index + 2;
        }
    }
    System.out.println("Gagal menambahkan data karena kapasitas tree array sudah penuh.");
}

```

traversePreOrder()

```

void traversePreOrder() {
    traversePreOrderHelper(0);
}

```

traversePreOrderHelper(int idxStart)

```

void traversePreOrderHelper(int idxStart) {
    if (idxStart <= idxLast) {
        if (dataMahasiswa[idxStart] != null) {
            dataMahasiswa[idxStart].tampilInformasi();
            traversePreOrderHelper(2 * idxStart + 1);
            traversePreOrderHelper(2 * idxStart + 2);
        }
    }
}

```